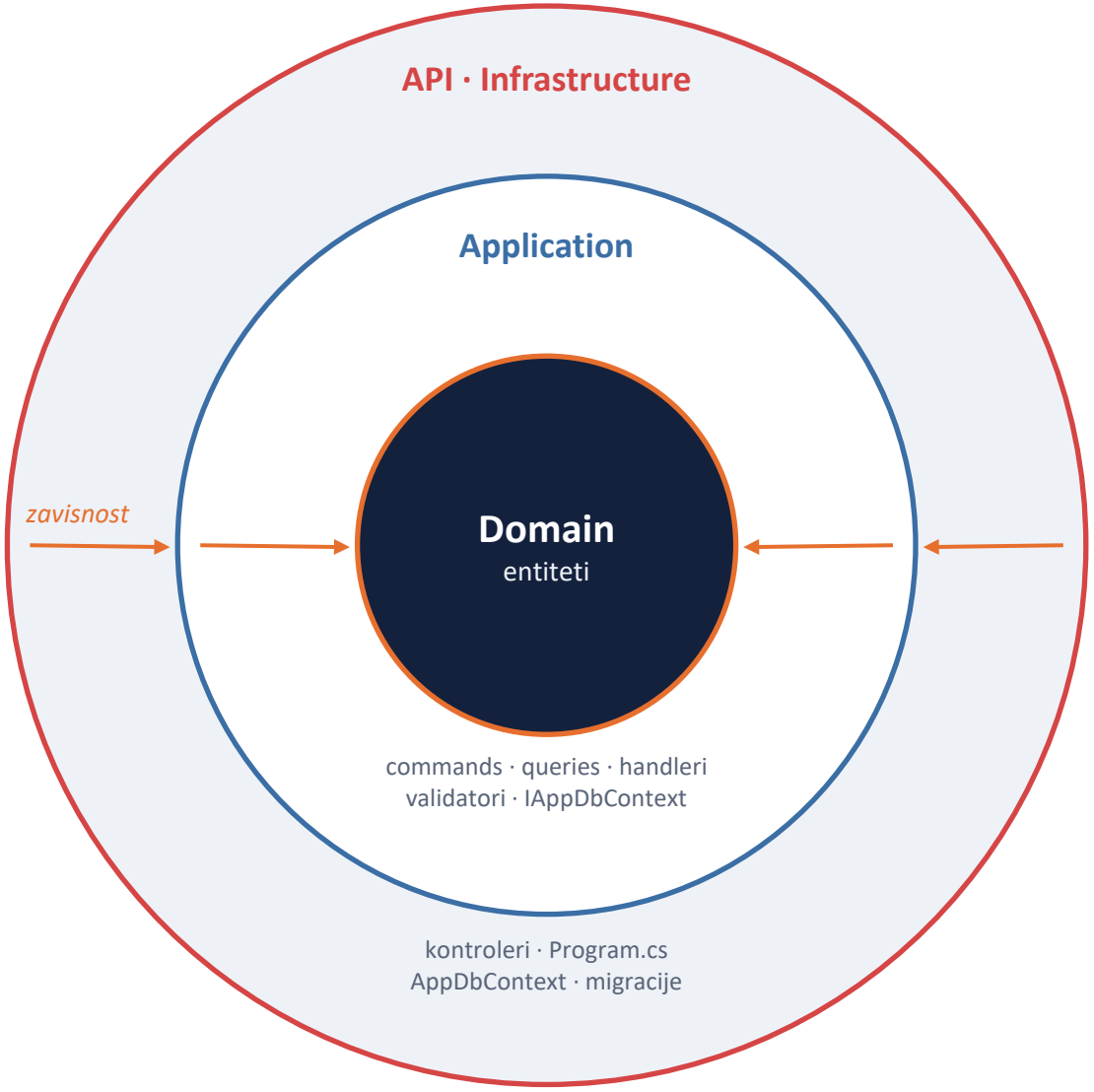
Clean Architecture i soft delete

Razvoj softvera I · Vježbe 5 · FIT UNMO 2026/27

Nadogradnja V04: četiri projekta, BaseEntity i global query filter

Clean Architecture: zavisnosti idu samo prema unutra



Domain: centar

Entiteti Student, Smjer, Predmet, Upis i BaseEntity. Ne zavisi ni od čega: nema EF Core, MediatR ni ASP.NET.

Application: šta aplikacija radi

Commands, queries, handleri, validatori, DTO i izuzeci. Bazi podataka pristupa preko interfejsa IAppDbContext, ne preko klase AppDbContext.

Infrastructure i API: kako

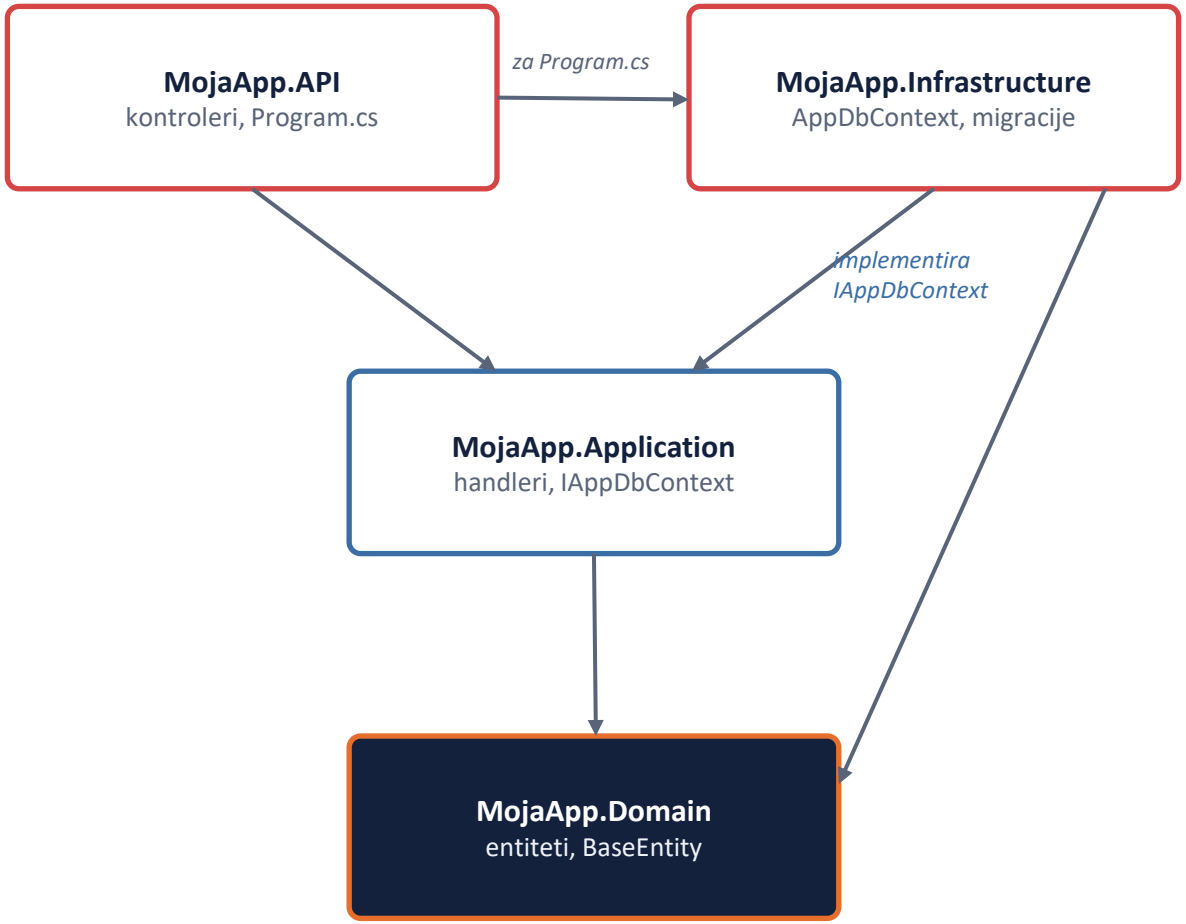
Infrastructure: EF Core, SQL Server, AppDbContext, migracije. API: HTTP, kontroleri, GlobalExceptionHandler, Program.cs.

Pravilo zavisnosti

unutrašnji sloj ne referencira spoljni

Application definiše IAppDbContext, a Infrastructure ga implementira. Zato handler radi s bazom podataka, a Application ne referencira Infrastructure.

Četiri projekta: reference u .csproj prate pravilo zavisnosti



Projekat	Referencira	NuGet paketi
Domain	—	—
Application	Domain	MediatR, FluentValidation (+ DI), EF Core
Infrastructure	Application, Domain	EF Core SqlServer
API	Application, Infrastructure	EF Core Tools, Swashbuckle

Starter za seminarski rad

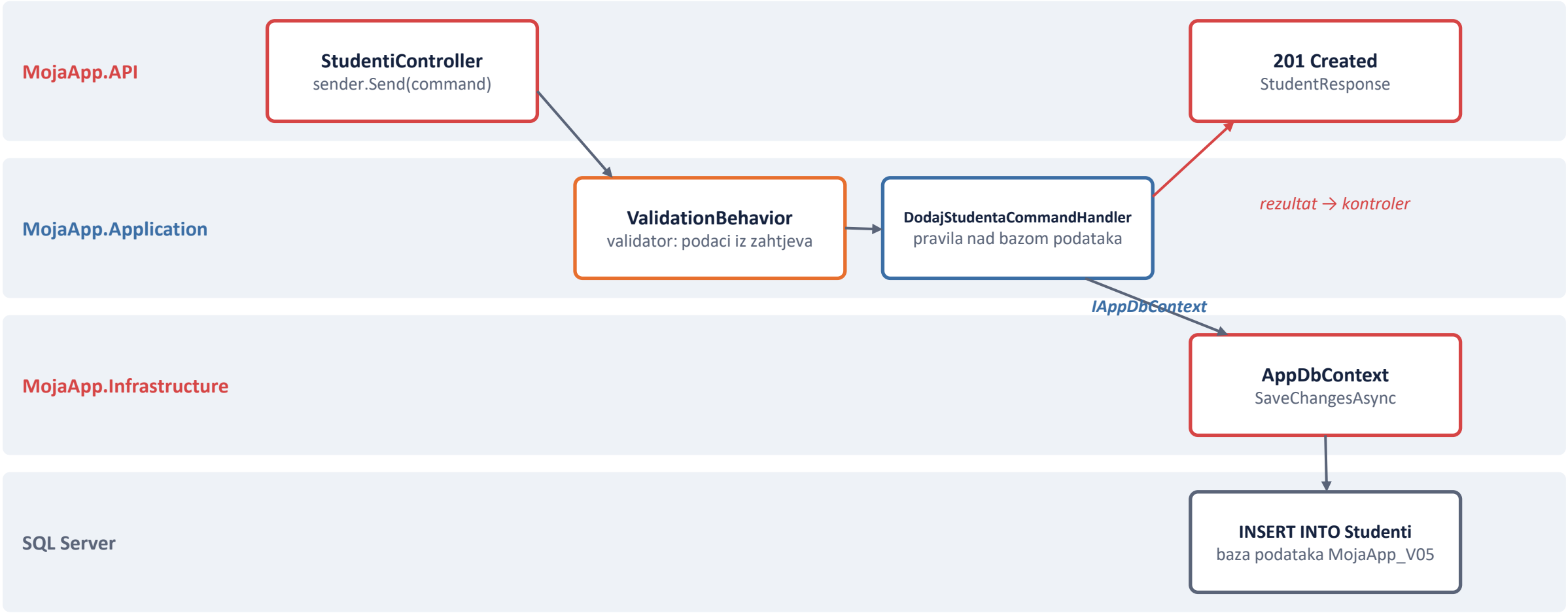
Isti raspored: Market.Domain, Market.Application, Market.Infrastructure, Market.API, i dodatno Market.Shared (konstante, opcije, zajednički DTO).

MediatR i FluentValidation u API-ju

paket se ne dodaje ponovo

API ih dobija preko reference na Application (paketi prelaze s referencom).

POST /Studenti prolazi kroz sva četiri projekta



Domain nema svoju traku: entitet Student koriste Application (handler ga pravi) i Infrastructure (AppDbContext ga snima).

Folderi prije i poslije: broj na kraju reda je korak u kojem se mijenja

folderi V04

```
MojaApp.sln
└─ MojaApp.API/
   ├─ Common/  Behaviors, Exceptions
   ├─ Controllers/
   ├─ Data/AppDbContext.cs
   ├─ Dtos/
   ├─ Middleware/GlobalExceptionHandler.cs
   ├─ Migrations/  (V02–V04)
   ├─ Models/
   ├─ Modules/  Studenti, Predmeti, Upisi
   ├─ GlobalUsings.cs
   └─ Program.cs
```

Šta nestaje

- Migrations iz V02–V04
  V05 počinje novu bazu podataka s jednom migracijom (korak 8).
- UpisDodajRequest i DatumKreiranja
  Zamjenjuju ih DodajUpisCommand (korak 5) i CreatedAtUtc iz BaseEntity (korak 6).

folderi V05

```
MojaApp.sln
├─ MojaApp.Domain/                          // 1
│   ├─ Common/BaseEntity.cs                 // 6
│   └─ Entities/  (Models iz V04)           // 2
├─ MojaApp.Application/                     // 1
│   ├─ Abstractions/IAppDbContext.cs        // 3
│   ├─ Common/ · Dtos/ · Modules/           // 3, 5
│   └─ DependencyInjection.cs               // 3
├─ MojaApp.Infrastructure/                  // 1
│   ├─ Database/AppDbContext.cs             // 4, 6, 7
│   ├─ Database/Configurations/             // 4, 7
│   ├─ Database/Migrations/                 // 8
│   ├─ DatabaseInitializer.cs               // 8
│   └─ DependencyInjection.cs               // 4
└─ MojaApp.API/
   ├─ Controllers/  samo ISender            // 5
   ├─ Middleware/GlobalExceptionHandler.cs
   └─ Program.cs                            // 3, 4, 8
```

Osam koraka: poslije svakog build, a poslije koraka 5 i 8 i .http

#	Korak	Šta se mijenja	Provjera
1	Projekti	tri Class Library projekta (.NET 8), reference, solution	build
2	Domain	Models → MojaApp.Domain/Entities, novi namespace	build
3	Application	Common, Dtos, Modules; paketi; IAppDbContext; AddApplication	build
4	Infrastructure	AppDbContext, Configurations, AddInfrastructure; brišu se stare migracije	build
5	API	kontroleri samo s ISender: Smjerovi, statistika, Upisi	pokretanje nad bazom podataka iz V04; .http
6	BaseEntity	Domain/Common/BaseEntity.cs; CreatedAtUtc i ModifiedAtUtc u SaveChangesAsync	build
7	Soft delete	Remove → UPDATE; global query filter; filtrirani indeksi	build
8	Baza podataka	migracija Pocetna; Migrate pri pokretanju; baza podataka MojaApp_V05	pokretanje; .http

terminal nova grana, pa commit poslije svakog koraka

```
git switch -c v05-clean-architecture
git add .
git commit -m "V05 korak 1: projekti"
```

Premještanje fajlova

u Visual Studiju: prevući folder u drugi projekat

Namespace se ne mijenja sam: u svakom premještenom fajlu se ispravlja (Ctrl+H, Replace in Files).

01

Četiri projekta: koraci 1–5

Novi projekti, premještanje koda sloj po sloj i kontroleri bez AppDbContext.

Ponašanje API-ja ostaje kao u V04.

Korak 1: tri nova projekta i reference između njih

1 Projekti | 2 Domain | 3 Application | 4 Infrastructure | 5 API | 6 BaseEntity | 7 Soft delete | 8 Baza podataka

cmd dotnet CLI, u folderu rješenja (gdje je MojaApp.sln); provjereno na .NET SDK 10

```
dotnet new classlib -n MojaApp.Domain -f net8.0
dotnet new classlib -n MojaApp.Application -f net8.0
dotnet new classlib -n MojaApp.Infrastructure -f net8.0
dotnet sln add MojaApp.Domain MojaApp.Application MojaApp.Infrastructure

dotnet add MojaApp.Application reference MojaApp.Domain
dotnet add MojaApp.Infrastructure reference MojaApp.Application MojaApp.Domain
dotnet add MojaApp.API reference MojaApp.Application MojaApp.Infrastructure
```

XML rezultat u MojaApp.API.csproj

```
<ProjectReference Include="..\MojaApp.Application\MojaApp.Application.csproj" />
<ProjectReference Include="..\MojaApp.Infrastructure\MojaApp.Infrastructure.csproj" />
```

Visual Studio

Solution → Add → New Project → Class Library

Framework .NET 8.0. Reference: desni klik na projekat → Add → Project Reference.

Provjera

build bez grešaka

Novi projekti su prazni, a API radi kao u V04. API preko Application i Infrastructure vidi i Domain.

Korak 2: Models prelaze u MojaApp.Domain/Entities

C# MojaApp.Domain/Entities/Smjer.cs

```
namespace MojaApp.Domain.Entities;

// Jedan smjer ima više studenata (1:N).
public class Smjer
{
    public int Id { get; set; }
    public string Naziv { get; set; } = "";

    // Navigacija na "više" stranu veze: svi studenti ovog
smjera.
    public List<Student> Studenti { get; set; } = [];
}
```

V04	V05
`namespace MojaApp.API.Models;`	`namespace MojaApp.Domain.Entities;`
`global using MojaApp.API.Models;`	`global using MojaApp.Domain.Entities;`

folderi folder Models iz API-ja se briše

```
MojaApp.Domain/
└─ Entities/
   ├─ Predmet.cs
   ├─ Smjer.cs
   ├─ Student.cs
   └─ Upis.cs
```

Domain nema paketa

samo klase i svojstva

Navigacije (List<Upis>, Smjer?) ostaju: to je C#, ne EF Core.

Provjera

build bez grešaka

Migracije iz V04 imaju stari namespace samo u tekstu (string), pa se i dalje kompajliraju.

Korak 3: Common, Dtos i Modules prelaze u Application, s paketima

1 Projekti | 2 Domain | **3 Application** | 4 Infrastructure | 5 API | 6 BaseEntity | 7 Soft delete | 8 Baza podataka

cmd dotnet CLI, u folderu rješenja

```
dotnet add MojaApp.Application package MediatR --version 12.5.0
dotnet add MojaApp.Application package FluentValidation --version 12.2.0
dotnet add MojaApp.Application package FluentValidation.DependencyInjectionExtensions --version 12.2.0
dotnet add MojaApp.Application package Microsoft.EntityFrameworkCore --version 8.0.21
dotnet remove MojaApp.API package MediatR    (isto za oba FluentValidation paketa)
```

PMC ili Package Manager Console: isti paketi, -ProjectName MojaApp.Application

```
Install-Package MediatR -Version 12.5.0 -ProjectName MojaApp.Application
```

C# MojaApp.Application/GlobalUsings.cs

```
global using FluentValidation;
global using MediatR;
global using Microsoft.EntityFrameworkCore;
global using MojaApp.Application.Abstractions;
global using MojaApp.Application.Common.Exceptions;
global using MojaApp.Application.Dtos;
global using MojaApp.Domain.Entities;
```

Zašto EF Core u Application

Handleri pišu LINQ upite (DbSet<T>, ToListAsync, AnyAsync). UseSqlServer ostaje u Infrastructure.

Provjera

build tek poslije sljedećeg slajda

Handleri još traže AppDbContext iz API-ja, a Application ne smije referencirati API.

Korak 3: handleri traže IAppDbContext umjesto AppDbContext

C# MojaApp.Application/Abstractions/IAppDbContext.cs

```
// Handleri rade s ovim interfejsom, a ne s AppDbContext klasom.
// Implementacija (AppDbContext, SQL Server) je u Infrastructure, pa Application ne zavisi od Infrastructure.
// Isto je u starteru: Market.Application/Abstractions/IAppDbContext.cs.
public interface IAppDbContext
{
    DbSet<Student> Studenti { get; }
    DbSet<Smjer> Smjerovi { get; }
    DbSet<Predmet> Predmeti { get; }
    DbSet<Upis> Upisi { get; }

    Task<int> SaveChangesAsync(CancellationToken ct);
}
```

C# handler: samo tip parametra

```
// V04
public class ObrisiStudentaCommandHandler(AppDbContext db)
// V05: isto u svim handlerima
public class ObrisiStudentaCommandHandler(IAppDbContext db)
```

AppDbContext u API-ju

… : DbContext(options), IAppDbContext

Do koraka 4 klasa ostaje u API-ju, samo dobija interfejs.

Korak 3: AddApplication registruje MediatR i validatore

C# MojaApp.Application/DependencyInjection.cs i Program.cs

```
public static IServiceCollection AddApplication(this IServiceCollection services)
{
    var assembly = typeof(DependencyInjection).Assembly; // ovaj projekat (MojaApp.Application)

    // MediatR: handleri su u ovom projektu, ne u API-ju.
    services.AddMediatR(cfg => cfg.RegisterServicesFromAssembly(assembly));

    // FluentValidation: svi validatori iz ovog projekta.
    services.AddValidatorsFromAssembly(assembly);

    // Prije svakog handlera MediatR pokreće validatore za taj zahtjev.
    services.AddTransient(typeof(IPipelineBehavior<,>), typeof(ValidationBehavior<,>));

    return services;
}

// Program.cs (do koraka 4; AddScoped prelazi u AddInfrastructure)
// Application: MediatR, validatori, ValidationBehavior.
builder.Services.AddApplication();

// Handler traži IAppDbContext, a dobija AppDbContext (jedan po zahtjevu).
builder.Services.AddScoped<IAppDbContext>(sp => sp.GetRequiredService<AppDbContext>());
```

Provjera: build bez grešaka. Bez AddScoped<IAppDbContext> aplikacija se ne pokreće (izmjereno; slajd Česte greške).

Korak 4: AppDbContext prelazi u Infrastructure

1 Projekti | 2 Domain | 3 Application | **4 Infrastructure** | 5 API | 6 BaseEntity | 7 Soft delete | 8 Baza podataka

C# MojaApp.Infrastructure/Database/AppDbContext.cs

```
namespace MojaApp.Infrastructure.Database;

// Most između C# koda i baze podataka: svaki DbSet odgovara jednoj tabeli.
// Implementira IAppDbContext iz Application, pa handleri ne znaju za ovu klasu ni za SQL Server.
public class AppDbContext(DbContextOptions<AppDbContext> options) : DbContext(options), IAppDbContext
{
    public DbSet<Student> Studenti => Set<Student>();
    public DbSet<Smjer> Smjerovi => Set<Smjer>();
    public DbSet<Predmet> Predmeti => Set<Predmet>();
    public DbSet<Upis> Upisi => Set<Upis>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Konfiguracija svakog entiteta je u svojoj klasi (folder Configurations), kao u starteru.
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }
}
```

Paket

- Microsoft.EntityFrameworkCore.SqlServer 8.0.21
  Dodaje se u Infrastructure, briše iz API-ja. Tools ostaje u API-ju (startup projekat za migracije).

GlobalUsings u Infrastructure

- Microsoft.EntityFrameworkCore, MojaApp.Domain.Entities
  Isto kao u Application: kraći fajlovi konfiguracija.

Korak 4: Fluent API iz OnModelCreating prelazi u klasu po entitetu

C# MojaApp.Infrastructure/Database/Configurations/PredmetConfiguration.cs

```
public class PredmetConfiguration : IEntityTypeConfiguration<Predmet>
{
    public void Configure(EntityTypeBuilder<Predmet> builder)
    {
        builder.ToTable("Predmeti");
        builder.Property(x => x.Naziv).HasMaxLength(100).IsRequired();
        builder.Property(x => x.Sifra).HasMaxLength(10).IsRequired();

        // Dva predmeta ne mogu imati istu šifru.
        builder.HasIndex(x => x.Sifra)
            .IsUnique();

        builder.HasData(
            new Predmet { Id = 1, Naziv = "Razvoj softvera I", Sifra = "RS1", Ects = 7
},
            new Predmet { Id = 2, Naziv = "Programiranje III", Sifra = "PR3", Ects = 6
},
            new Predmet { Id = 3, Naziv = "Baze podataka II", Sifra = "BP2", Ects = 6
```

Četiri klase

- Smjer, Student, Predmet, Upis

  Folder Database/Configurations, po jedna klasa za svaki entitet.

HasData ostaje uz entitet

- početni podaci iz V03

  Smjerovi, studenti, predmeti i upisi, svaki u svojoj konfiguraciji.

Korak 4: AddInfrastructure registruje AppDbContext i IAppDbContext

C# MojaApp.Infrastructure/DependencyInjection.cs i Program.cs

```
public static IServiceCollection AddInfrastructure(this IServiceCollection services, IConfiguration configuration)
{
    // Baza podataka: connection string "Main" iz appsettings.json (projekat MojaApp.API).
    services.AddDbContext<AppDbContext>(options =>
        options.UseSqlServer(configuration.GetConnectionString("Main")));

    // Handler traži IAppDbContext, a dobija isti AppDbContext (jedan po zahtjevu).
    services.AddScoped<IAppDbContext>(sp => sp.GetRequiredService<AppDbContext>());

    return services;
}

// Program.cs: umjesto AddDbContext, AddMediatR, AddValidators… iz V04
// Application: MediatR, validatori, ValidationBehavior.
builder.Services.AddApplication();

// Infrastructure: AppDbContext (SQL Server) i IAppDbContext.
builder.Services.AddInfrastructure(builder.Configuration);
```

Stare migracije se brišu

- folder Migrations iz API-ja

  Nova migracija Pocetna je u koraku 8.

Jedan objekat, dva tipa

- AddScoped<IAppDbContext>(…)

  Handler i AppDbContext dijele isti objekat u zahtjevu.

Provjera

build bez grešaka

Smjerovi i Upisi još koriste AppDbContext (korak 5).

Korak 5: kontroleri bez AppDbContext, sve ide kroz MediatR

1 Projekti | 2 Domain | 3 Application | 4 Infrastructure | **5 API** | 6 BaseEntity | 7 Soft delete | 8 Baza podataka

Endpoint	Query ili command (Application/Modules)
GET /Smjerovi	Smjerovi/Queries/Lista: ListaSmjerovaQuery
GET /Predmeti/5/statistika	Predmeti/Queries/Statistika: PredmetStatistikaQuery
GET /Upisi/5	Upisi/Queries/GetById: UpisGetByIdQuery
POST /Upisi	Upisi/Commands/Dodaj: DodajUpisCommand + validator
DELETE /Upisi/5	Upisi/Commands/Obrisi: ObrisiUpisCommand

Provjera

pokretanje nad bazom podataka iz V04

Izmjereno: 37/37 zahtjeva iz .http. Statusi kao u V04, osim upisa nepostojećeg studenta ili predmeta: 404 umjesto 400.

C# Controllers/UpisiController.cs: isto kao Dodaj za Predmete u V04

```
// POST /Upisi: upis studenta na predmet (201; duplikat 409; student ili predmet ne postoji 404)
[HttpPost]
public async Task<ActionResult<UpisResponse>> Dodaj(DodajUpisCommand command, CancellationToken ct)
{
    int id = await sender.Send(command, ct);
    UpisResponse upis = await sender.Send(new UpisGetByIdQuery { Id = id }, ct);
    return CreatedAtAction(nameof(GetById), new { id }, upis);
}
```

V04 je postojanje studenta i predmeta provjeravao u kontroleru (ModelState, 400). Po pravilu iz V04 (validator bez baze podataka) to je sada u DodajUpisCommandHandler, pa je status 404.

02

BaseEntity i soft delete: koraci 6–8

Zajednička polja za sve entitete, brisanje bez DELETE i filter koji skriva obrisane zapise.

Na kraju nova baza podataka s jednom migracijom.

Korak 6: BaseEntity nosi Id, IsDeleted i vrijeme kreiranja i izmjene

C# MojaApp.Domain/Common/BaseEntity.cs

```
namespace MojaApp.Domain.Common;

// Zajednička polja svih entiteta (kao Market.Domain/Common/BaseEntity.cs u starteru).
// Vrijednosti CreatedAtUtc, ModifiedAtUtc i IsDeleted postavlja AppDbContext pri SaveChanges, ne handler.
public class BaseEntity
{
    public int Id { get; set; }

    // Soft delete: zapis se ne briše iz tabele, nego dobija IsDeleted = true.
    public bool IsDeleted { get; set; }

    public DateTime CreatedAtUtc { get; set; }
    public DateTime? ModifiedAtUtc { get; set; } // null dok zapis nije mijenjan
}
```

C# MojaApp.Domain/Entities/Student.cs

```
public class Student : BaseEntity
{
    public string Ime { get; set; } = "";
    …   // ostala svojstva kao ranije, bez Id i DatumKreiranja
}
```

Isto za Smjer, Predmet, Upis

- public class Predmet : BaseEntity

  U starteru: Market.Domain/Common/BaseEntity.cs, ista četiri svojstva.

Korak 6: SaveChangesAsync sam postavlja CreatedAtUtc i ModifiedAtUtc

1 Projekti | 2 Domain | 3 Application | 4 Infrastructure | 5 API | **6 BaseEntity** | 7 Soft delete | 8 Baza podataka

C# MojaApp.Infrastructure/Database/AppDbContext.cs (korak 6)

```
// Prije svakog snimanja: vrijeme kreiranja i izmjene.
// Isto radi starter (Market.Infrastructure/Database/DatabaseConfiguration.cs).
public override Task<int> SaveChangesAsync(CancellationToken cancellationToken = default)
{
    DateTime sada = DateTime.UtcNow;

    foreach (var entry in ChangeTracker.Entries<BaseEntity>())
    {
        switch (entry.State)
        {
            case EntityState.Added:
                entry.Entity.CreatedAtUtc = sada;
                entry.Entity.IsDeleted = false;
                break;

            case EntityState.Modified:
                entry.Entity.ModifiedAtUtc = sada;
                break;
        }
    }

    return base.SaveChangesAsync(cancellationToken);
}
```

Početni podaci

- HasData ne ide kroz SaveChangesAsync
  Zato HasData dobija fiksno CreatedAtUtc = Seed.Datum (Configurations/Seed.cs).

U handleru

- red DatumKreiranja = … se briše
  DodajStudentaCommandHandler

Provjera

build bez grešaka

Šema je nova: aplikacija se pokreće tek u koraku 8.

Korak 7: Remove ne briše red, nego postavlja IsDeleted = 1

C# AppDbContext.SaveChangesAsync: novi case

```
case EntityState.Deleted:
    // Remove(...) u handleru: umjesto DELETE ide UPDATE IsDeleted = 1, a red ostaje u tabeli.
    entry.State = EntityState.Modified;
    entry.Entity.IsDeleted = true;
    entry.Entity.ModifiedAtUtc = sada;
    break;
```

SQL DELETE /Predmeti/4: handler zove Remove, u logu je UPDATE

```
UPDATE [Predmeti] SET [CreatedAtUtc] = @p0, [Ects] = @p1,
  [IsDeleted] = @p2, [ModifiedAtUtc] = @p3, [Naziv] = @p4,
  [Sifra] = @p5
OUTPUT 1
WHERE [Id] = @p6;
```

SQL tabela Predmeti poslije testa

Id	Sifra	IsDeleted	ModifiedAtUtc
4	ASP	1	2026-11-04 01:50:36
5	ASP	0	NULL

Zašto soft delete

- podaci ostaju za historiju i vraćanje
  Red ostaje u tabeli i može se vratiti (samostalna vježba 3). U testu u logu nema nijednog DELETE FROM.

Korak 7: global query filter dodaje WHERE IsDeleted = 0 svakom upitu

C# MojaApp.Infrastructure/Database/AppDbContext.cs

```
// u OnModelCreating, poslije ApplyConfigurationsFromAssembly: PrimijeniGlobalniFilter(modelBuilder);

private static void PrimijeniGlobalniFilter(ModelBuilder modelBuilder)
{
    foreach (var entityType in modelBuilder.Model.GetEntityTypes())
    {
        if (!typeof(BaseEntity).IsAssignableFrom(entityType.ClrType))
            continue;

        // Pravi lambdu e => e.IsDeleted == false za konkretan tip (Student, Predmet…).
        var e = Expression.Parameter(entityType.ClrType, "e");
        var uslov = Expression.Equal(
            Expression.Property(e, nameof(BaseEntity.IsDeleted)),
            Expression.Constant(false));

        modelBuilder.Entity(entityType.ClrType).HasQueryFilter(Expression.Lambda(uslov, e));
    }
}
```

Jedan entitet

- builder.HasQueryFilter(x => !x.IsDeleted)
  U konfiguraciji entiteta. Petlja radi isto za sve entitete, kao u starteru.

Zaobilaženje filtera

db.Predmeti.IgnoreQueryFilters()

Samo kad se obrisani zapisi traže namjerno (samostalne vježbe 2 i 3).

Korak 7: filter je u SQL-u, i u JOIN-u prema smjeru

SQL GET /Studenti, iz loga (prelomljeno)

```
SELECT [s].[Id], [s].[Ime] + N' ' + [s].[Prezime] AS [ImePrezime],
       [s].[BrojIndeksa], [s].[DatumRodjenja], [t].[Naziv] AS [Smjer]
FROM [Studenti] AS [s]
LEFT JOIN (
    SELECT [s0].[Id], [s0].[Naziv]
    FROM [Smjerovi] AS [s0]
    WHERE [s0].[IsDeleted] = CAST(0 AS bit)
) AS [t] ON [s].[SmjerId] = [t].[Id]
WHERE [s].[IsDeleted] = CAST(0 AS bit)
ORDER BY [s].[Prezime], [s].[Ime]
```

Isti handleri kao u V04: nijedan red koda u Application nije mijenjan zbog filtera (samo komentari).

Gdje se vidi SQL

appsettings.Development.json, kao u V03

"Microsoft.EntityFrameworkCore.Database.Command": "Information" ispisuje svaki upit u konzolu.

Obrisano se ponaša kao nepostojeće

- GET /Predmeti/4 poslije brisanja → 404
- AnyAsync u handlerima ne vidi obrisane
  Zato se šifra obrisanog predmeta može ponovo upotrijebiti (sljedeći slajd).
- x.Upisi.Count ne broji obrisane upise
  Statistika predmeta i broj studenata po smjeru rade bez izmjene.
- obrisan smjer studentu daje Smjer = null
  LEFT JOIN na podupit s filterom.

Korak 7: jedinstveni indeksi i provjere u handlerima

C# PredmetConfiguration.cs (isto za Upis: StudentId, PredmetId)

```
// Dva neobrisana predmeta ne mogu imati istu šifru.
// Filter: obrisani (IsDeleted = 1) predmet ostaje u tabeli, pa bi bez filtera
// njegova šifra ostala zauzeta i novi predmet s istom šifrom bi pao na indeksu.
builder.HasIndex(x => x.Sifra)
    .IsUnique()
    .HasFilter("[IsDeleted] = 0");
```

C# ObrisiPredmetCommandHandler.cs (provjera iz V04, novi komentar)

```
// Soft delete je UPDATE, pa OnDelete(Restrict) na Upis više ne sprječava brisanje.
// Ova provjera je zato jedina zaštita.
if (await db.Upisi.AnyAsync(x => x.PredmetId == request.Id, ct))
    throw new ConflictException("Predmet ima upisane studente i ne može se obrisati."); // 409
```

Indeks bez HasFilter

- obriše se ASP, pa novi predmet ASP → **500**

  Log: Cannot insert duplicate key row in object 'dbo.Predmeti' with unique index 'IX_Predmeti_Sifra'.

Bez provjere upisa u handleru

- DELETE /Predmeti/1 s upisima → **204**

  Restrict ne djeluje na UPDATE. Poslije toga GET /Upisi/1 → 404, a upis nestaje iz detalja studenta.

Korak 8: nova baza podataka, jedna migracija i Migrate pri pokretanju

1 Projekti | 2 Domain | 3 Application | 4 Infrastructure | 5 API | 6 BaseEntity | 7 Soft delete | **8 Baza podataka**

PMC Package Manager Console

```
Add-Migration Pocetna -Project MojaApp.Infrastructure -StartupProject MojaApp.API -OutputDir Database\Migrations
```

cmd ili dotnet CLI, u folderu rješenja (-o je kraće ime za --output-dir)

```
dotnet ef migrations add Pocetna --project MojaApp.Infrastructure --startup-project MojaApp.API -o Database/Migrations
```

C# MojaApp.Infrastructure/DatabaseInitializer.cs

```
public static async Task InitializeDatabaseAsync(this IServiceProvider services)
{
    await using var scope = services.CreateAsyncScope();
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    await db.Database.MigrateAsync();
}

// Program.cs, odmah poslije var app = builder.Build();
await app.Services.InitializeDatabaseAsync();
```

appsettings.json

- Database=MojaApp_V05
  Nova baza podataka; stara iz V04 ostaje netaknuta.

Log pri prvom pokretanju

- CREATE DATABASE [MojaApp_V05]
  Applying migration '20261104014921_Pocetna'.

Provjera

Update-Database nije potreban

MigrateAsync napravi bazu podataka i primijeni migracije. Isto radi starter.

Korak 8: provjera soft delete-a zahtjevima iz .http fajla

1 Projekti | 2 Domain | 3 Application | 4 Infrastructure | 5 API | 6 BaseEntity | 7 Soft delete | **8 Baza podataka**

Zahtjev	Status	Zašto
POST /Predmeti (šifra ASP)	201	novi predmet
DELETE /Predmeti/4	204	UPDATE IsDeleted = 1
GET /Predmeti/4	404	global query filter
DELETE /Predmeti/4 (ponovo)	404	obrisan se ne vidi ni handleru
POST /Predmeti (ista šifra ASP)	201	indeks s HasFilter
DELETE /Upisi/4, pa isti upis ponovo	204, 201	ponovni upis nakon ispisa
DELETE /Predmeti/1 (ima upise)	409	provjera u handleru
DELETE /Studenti/3, pa POST /Upisi za njega	204, 404	obrisan student ne postoji za upis

U tabeli ostaje sve

- Predmeti: dva reda sa šifrom ASP

  Jedan s IsDeleted = 1, drugi s 0. SQL Server Object Explorer ili SSMS.

Ponašanje kao u V04

svi raniji zahtjevi: isti statusi

Jedina razlika od V04: upis nepostojećeg studenta ili predmeta je 404 (korak 5).

03

Česte greške, starter i samostalne vježbe

Izmjerene greške pri nadogradnji, isti fajlovi u starteru i četiri samostalne vježbe.

Česte greške: simptom, uzrok i korak u kojem se ispravljaju

Simptom (poruka iz izlaza ili loga)	Uzrok	Korak
Build: error MSB4006: There is a circular dependency in the target dependency graph …	Application referencira Infrastructure	1
Aplikacija se ne pokreće: Unable to resolve service for type 'MojaApp.Application.Abstractions.IAppDbContext' while attempting to activate '…Handler'.	nema AddScoped<IAppDbContext>(…)	3, 4
Add-Migration: Your target project 'MojaApp.API' doesn't match your migrations assembly 'MojaApp.Infrastructure'.	migracija se pravi u API-ju (Default project)	8
Add-Migration: Your startup project 'MojaApp.Infrastructure' doesn't reference Microsoft.EntityFrameworkCore.Design.	startup projekat nije MojaApp.API	8
Novi predmet sa šifrom obrisanog predmeta → 500; Cannot insert duplicate key row … 'IX_Predmeti_Sifra'.	indeks bez HasFilter("[IsDeleted] = 0")	7
DELETE /Predmeti/1 s upisima → 204; upis poslije toga ne postoji (GET /Upisi/1 → 404).	nema provjere upisa u handleru	7

MojaApp i starter: isti fajlovi, starter ima više detalja

MojaApp V05	Starter (rs1-2026-27/starter/Market.Backend)	Razlika
Domain/Common/BaseEntity.cs	Market.Domain/Common/BaseEntity.cs	ista četiri svojstva
Application/Abstractions/IAppDbContext.cs	Market.Application/Abstractions/IAppDbContext.cs	više DbSet-ova
Infrastructure/Database/AppDbContext.cs	Market.Infrastructure/Database/DatabaseContext.cs i DatabaseConfiguration.cs	starter: TimeProvider umjesto DateTime.UtcNow; DateTime se čita kao UTC
Infrastructure/Database/Configurations/	Market.Infrastructure/Database/Configurations/	jedinstveni indeksi s HasFilter("[IsDeleted] = 0")
DependencyInjection.cs (Application, Infrastructure)	isti nazivi metoda: AddApplication, AddInfrastructure	starter registruje i JWT, trenutnog korisnika, konverter valuta
Infrastructure/DatabaseInitializer.cs	Market.Infrastructure/DatabaseInitializer.cs	starter: za testove InMemory i EnsureCreated; zatim početni podaci
—	Market.Shared	konstante, opcije, zajednički DTO

U seminarskom radu

novi entitet nasljeđuje BaseEntity i dobija konfiguraciju u Market.Infrastructure/Database/Configurations

Soft delete i filter rade sami; jedinstveni indeks treba HasFilter, a povezane zapise čuva provjera u handleru (409).

Samostalne vježbe: četiri zadatka nad rješenjem V05

1 · Nastavnik u četiri projekta

- BaseEntity, konfiguracija, migracija Nastavnici

  GET po Id-u, POST, DELETE (soft delete). Email jedinstven samo među neobrisanim: nakon brisanja isti email → 201.

2 · Obrisani predmeti

- GET /Predmeti/obrisani

  IgnoreQueryFilters() i Where(x => x.IsDeleted). Bez IgnoreQueryFilters lista je uvijek prazna.

3 · Vraćanje obrisanog predmeta

- PUT /Predmeti/{id}/vrati

  204; 404 ako predmet nije obrisan; 409 ako u međuvremenu postoji aktivan predmet s istom šifrom.

4 · Brisanje smjera

- DELETE /Smjerovi/{id}

  204; 409 ako smjer ima studente (Restrict ne pomaže kod soft delete-a); 404 ako ne postoji.

Rad ide kao u V02–V04: issue, grana, commit po koraku i pull request.